

HDOC - Day 8

Question 1: Find the radius, diameter, surface area and volume of a sphere when its radius is given in centimetres and the output is required in metres.

1. Understand the Problem

Input: Radius of sphere in centimetres (r_cm).

Output: Radius and diameter in metres, surface area in square metres, and volume in cubic metres.

Formula: $r = r_cm / 100$; diameter = $2r$; surface area = $4 \times \pi \times r \times r$; volume = $(4/3) \times \pi \times r \times r \times r$. Use $\pi = 3.1416$.

2. Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	r = 100 cm	r=1.0000 m; d=2.0000 m; A=12.5664 m ² ; V=4.1888 m ³	Same as expected	Pass
2	r = 50 cm	r=0.5000 m; d=1.0000 m; A=3.1416 m ² ; V=0.5236 m ³	Same as expected	Pass
3	r = 10 cm	r=0.1000 m; d=0.2000 m; A=0.1257 m ² ; V=0.0042 m ³	Same as expected	Pass

3. Algorithm

1. Start.
2. Read radius in centimetres, r_cm.
3. Convert radius to metres: $r = r_cm / 100$.
4. Calculate diameter = $2 \times r$.
5. Calculate surface area = $4 \times \pi \times r \times r$.
6. Calculate volume = $(4/3) \times \pi \times r \times r \times r$.
7. Display radius, diameter, surface area and volume.
8. Stop.

Algorithm Evaluation

For all three prepared test cases, the algorithm converts centimetres to metres first and then applies the sphere formulae. The calculated values match the expected outputs.

4. Flowchart

START -> Input r_cm -> $r = r_cm / 100$ -> $d = 2r$ -> $A = 4\pi r^2$ -> $V = (4/3)\pi r^3$ -> Display r, d, A, V -> **STOP**

The same flow is followed for each test case; therefore the flowchart produces the expected values for all prepared inputs.

5. C Program

```
#include <stdio.h>
#define PI 3.1416

int main()
{
    double r_cm, r, diameter, area, volume;
    printf("Enter radius of sphere in cm: ");
    scanf("%lf", &r_cm);
    r = r_cm / 100.0;
    diameter = 2 * r;
    area = 4 * PI * r * r;
    volume = (4.0 / 3.0) * PI * r * r * r;
    printf("Radius = %.4lf m\n", r);
    printf("Diameter = %.4lf m\n", diameter);
    printf("Surface Area = %.4lf sq.m\n", area);
    printf("Volume = %.4lf cu.m\n", volume);
    return 0;
}
```

6-7. Compilation, Execution and Evaluation

```
puppie@Anvikas-MacBook-Air Desktop % clang sphere.c -o sphere
puppie@Anvikas-MacBook-Air Desktop % ./sphere
Enter radius of sphere in cm: 100
Radius = 1.0000 m
Diameter = 2.0000 m
Surface Area = 12.5664 sq.m
Volume = 4.1888 cu.m
puppie@Anvikas-MacBook-Air Desktop %
```

```
puppie@Anvikas-MacBook-Air Desktop % ./sphere
Enter radius of sphere in cm: 50
Radius = 0.5000 m
Diameter = 1.0000 m
Surface Area = 3.1416 sq.m
Volume = 0.5236 cu.m
puppie@Anvikas-MacBook-Air Desktop %
```

```
puppie@Anvikas-MacBook-Air Desktop % ./sphere
Enter radius of sphere in cm: 10
Radius = 0.1000 m
Diameter = 0.2000 m
Surface Area = 0.1257 sq.m
Volume = 0.0042 cu.m
puppie@Anvikas-MacBook-Air Desktop %
```

Result: The program was compiled and executed successfully. All three test cases produced outputs matching the expected results.

HDOC - Day 8

Question 2: Find the image of a point after reflection in the x-axis.

1. Understand the Problem

Input: Coordinates (x, y) of a point.

Output: Coordinates of its image after reflection in the x-axis.

Formula: Reflection in the x-axis changes (x, y) to (x, -y). The x-coordinate remains unchanged and the sign of the y-coordinate is reversed.

2. Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	(3, 5)	(3, -5)	(3, -5)	Pass
2	(-4, 7)	(-4, -7)	(-4, -7)	Pass
3	(6, 0)	(6, 0)	(6, 0)	Pass

3. Algorithm

1. Start.
2. Read x and y coordinates.
3. Keep x unchanged.
4. Calculate reflected y-coordinate as $y_image = -y$.
5. Display the original point and reflected point (x, y_image).
6. Stop.

Algorithm Evaluation

For positive, negative and zero coordinate cases, the algorithm keeps x unchanged and reverses only the sign of y. The results match the expected outputs.

4. Flowchart

START -> Input x, y -> $y_image = -y$ -> Display (x, y_image) -> **STOP**

The flowchart correctly maps every point (x, y) to (x, -y), including the boundary case $y = 0$.

5. C Program

```
#include <stdio.h>

int main()
{
    int x, y, image_y;
    printf("Enter x-coordinate: ");
    scanf("%d", &x);
    printf("Enter y-coordinate: ");
    scanf("%d", &y);
    image_y = -y;
    printf("Original point: (%d, %d)\n", x, y);
    printf("Image after reflection in x-axis: (%d, %d)\n", x, image_y);
    return 0;
}
```

6-7. Compilation, Execution and Evaluation

```
puppie@Anvikas-MacBook-Air Desktop % clang reflection.c -o reflection
puppie@Anvikas-MacBook-Air Desktop % ./reflection
Enter x-coordinate: 3
Enter y-coordinate: 5
Original point: (3, 5)
Image after reflection in x-axis: (3, -5)
puppie@Anvikas-MacBook-Air Desktop %
```

```
puppie@Anvikas-MacBook-Air Desktop % ./reflection
Enter x-coordinate: -4
Enter y-coordinate: 7
Original point: (-4, 7)
Image after reflection in x-axis: (-4, -7)
puppie@Anvikas-MacBook-Air Desktop %
```

```
puppie@Anvikas-MacBook-Air Desktop % ./reflection
Enter x-coordinate: 6
Enter y-coordinate: 0
Original point: (6, 0)
Image after reflection in x-axis: (6, 0)
puppie@Anvikas-MacBook-Air Desktop %
```

Result: The reflection program was compiled and executed successfully. All prepared test cases passed.